

04 潜在空间

作者 NILUSHANAN KULASINGHAM

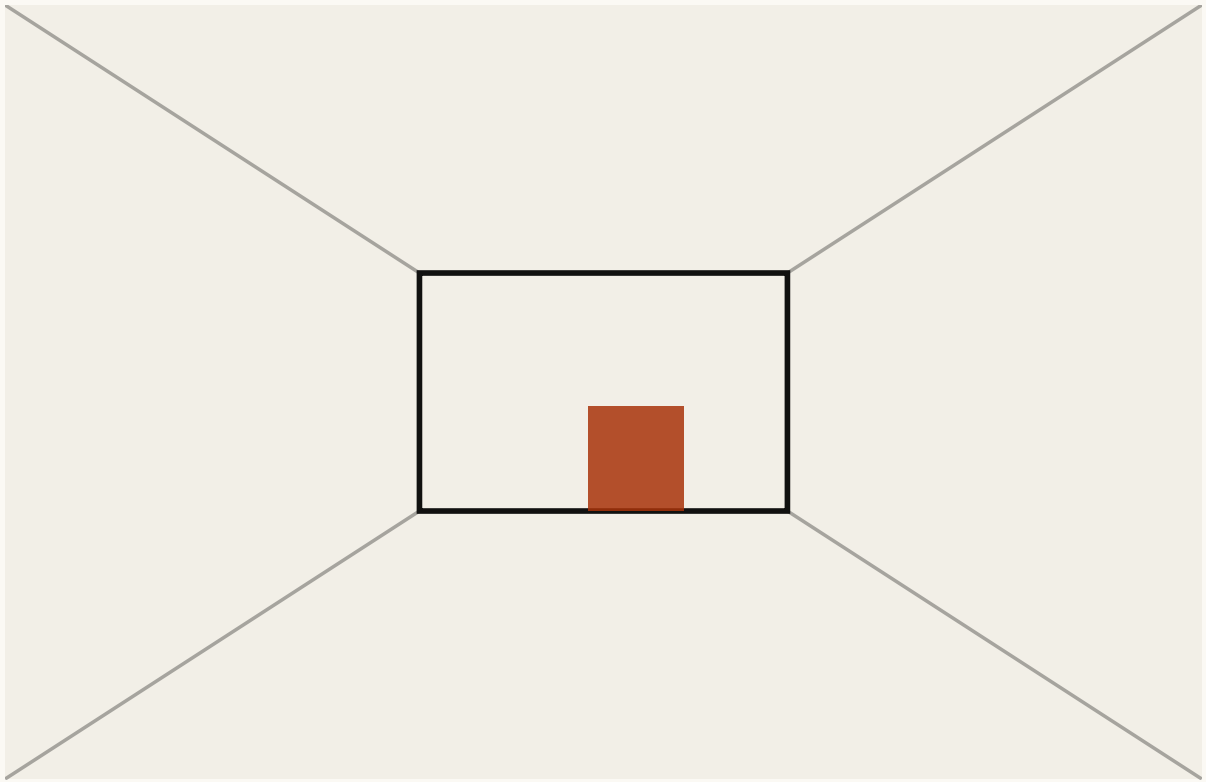
阅读约 12 分钟 · 可交互

摄像机测量几万个数字，而这个决定只需要两三个。中间那道收窄口发生了什么，以及为什么那个窄点给它后面的一切定了上限。

站在门口往房间里看。你的眼睛每秒钟大概在接收一亿次测量。

现在决定要不要往前走。

这些你几乎一样都没用上。远处那面墙有多远。有没有能过去的口子，大概在哪儿。这就是两三个数字，而你的眼睛收集到的其余一切，对这个决定来说都是装饰。



这两个数解码出来的房间

这个空间本身。在里面任意拖动	
墙有多远	0.45
能过去的口子在哪儿	0.62
画面里的数字个数	描述里的数字个数
74,800	2

FIG. 4.1 在右边那个方块里随便拖。它里面的每一个点都解码成一个房间，而这个房间是从那两个数字算出来的，不是查出来的。画面是几万个数字。产生它的那个东西是两个。

这两三个数字有个名字。它们是这个场景的**潜在** (latent 的意思是隐藏的、不直接可见的：这些数字从来不会展示给你看，没有任何东西给它们贴标签，但画面之所以长成那样，正是因为它们) 描述，而这两个数量之间的差距，就是这一章要讲的东西。在摄像机和决定之间，某个地方必须有一道**收窄口**。从另一头出来的东西，就是系统其余部分能拿到的全部。

那道收窄口

在系统中间放一个窄口，让所有东西都从那儿挤过去。

整个架构就是这样。一边是把画面拿进来、产出一串短数字的东西；另一边是拿着这串短数字、试着把画面重建出来的东西。把这两半一起训练，那么唯一能成功的办法，就是让这串短数字装下画面真正是由什么构成的。

这两半各有名字，值得记住，因为每篇论文都在用：前一半叫**编码器**，后一半叫**解码器**，而中间那段窄的叫**瓶颈**。

瓶颈会施加压力，却不会替坐标轴命名。许多同样紧凑的坐标系都能重建同一批画面；如果架构、数据或任务没有提供偏置，就没有什么能保证某一个坐标表示距离、另一个表示光照。Locatello 等人证明了更硬的边界。没有这些假设，纯无监督的解耦在一般情况下不可能实现。

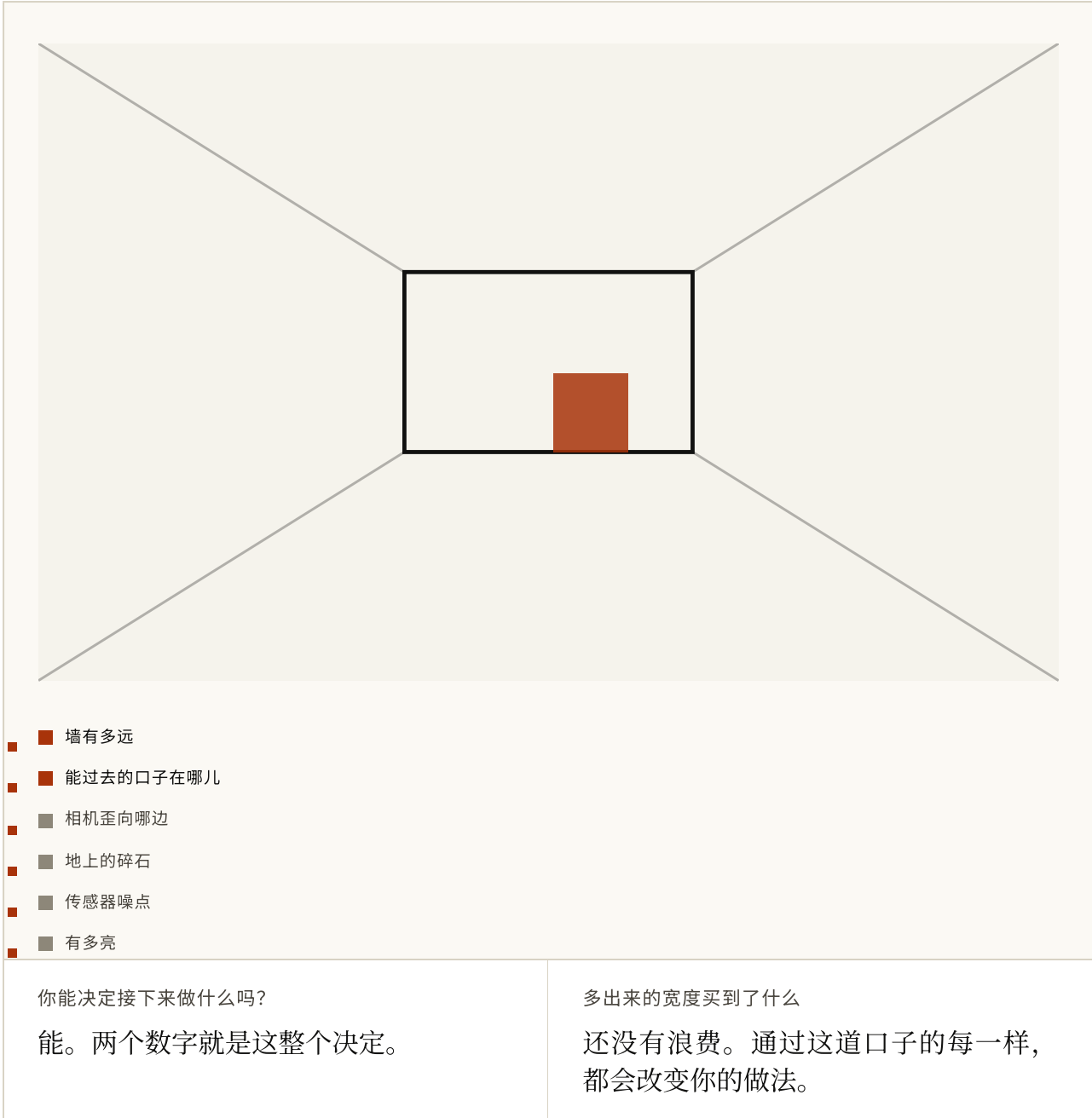


FIG. 4.2 这里的场景由六个数字生成，而这道口子只放前几个通过。把它从零往上拖。放进去两个数，决定就已经定下来了；再往后的都是碎石和传感器噪点，它们是真实的，也完全不会改变你的做法。

这幅图动过手脚，和任务有关的因素正好占据前两个坐标。学出来的系统不会白拿到这个顺序。「窄」逼出的是取舍；先买什么，由损失函数决定。重建损失很可能把预算花在最难画回来的东西上，而不是决定真正需要的东西上。

这不是思想实验。Ha 与 Schmidhuber 的世界模型前端放的就是这个形状的一道口子。一款赛车游戏的每一帧视频都被压成三十二个数字，之后的一切都只用这些数字工作。PlaNet 从原始像素做同样的事，然后就在出来的东西里做规划，从头到尾没有为了做决定而重建过任何一帧画面。

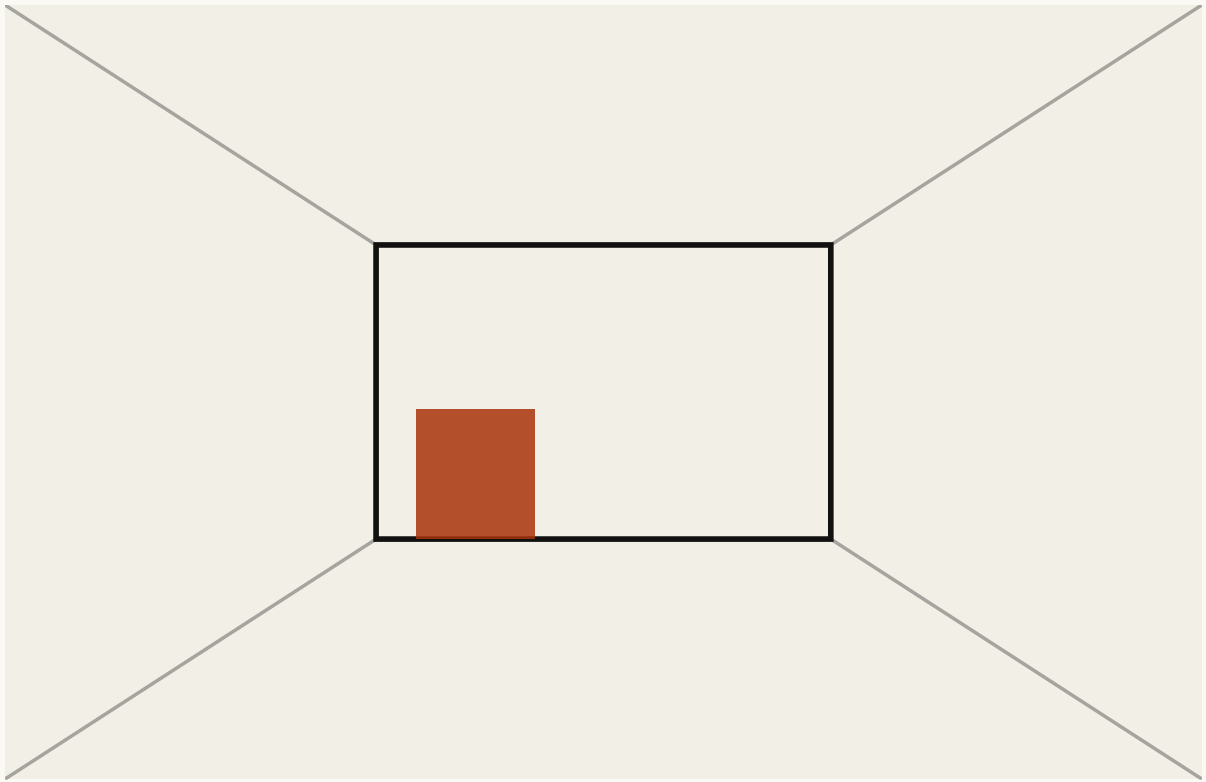
三十二个数字，而画面进来时是几万个。这个比例就是重点，而且它大致就是「摄像机测量了什么」和「一个决定取决于什么」之间的比例。

它是一个地方，不是一份清单

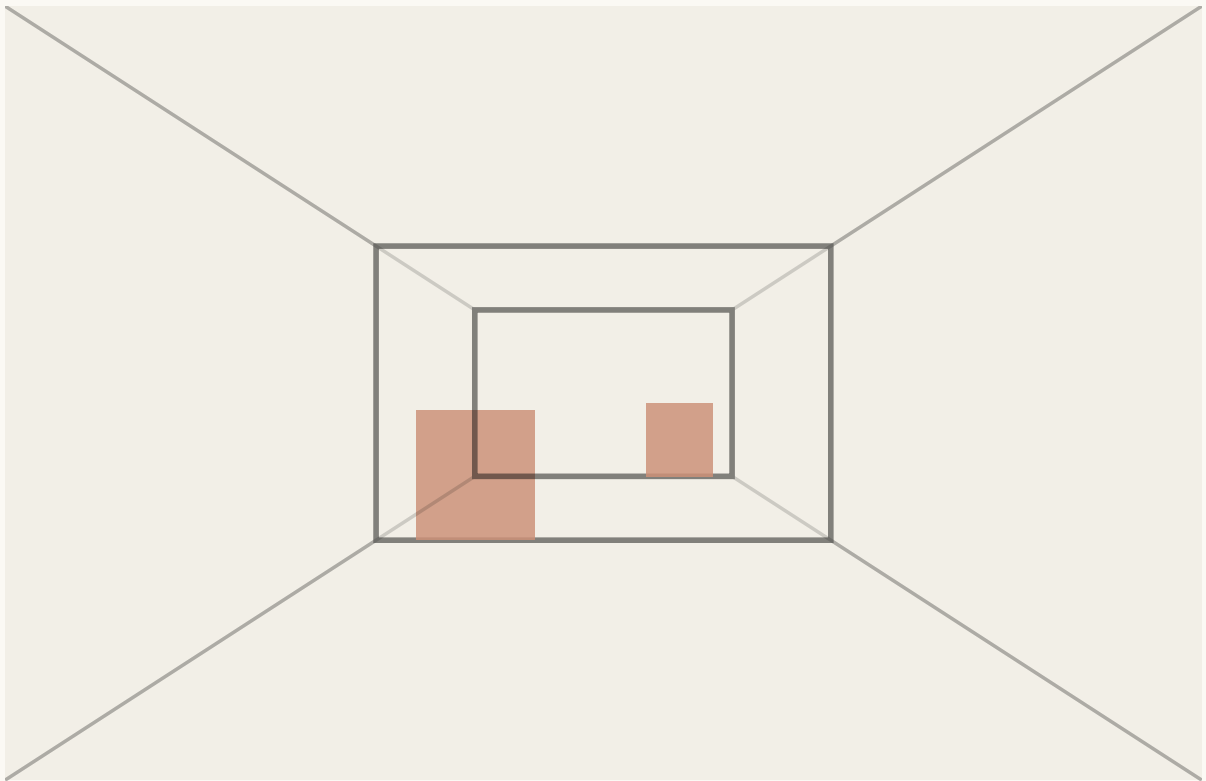
接下来这个转变要过一会儿才落得下去。

那些数字是坐标。在一个正则化得当的潜在空间里，它们指定的点会有有用的邻居、方向和距离。一个普通自编码器也可能学出一张扭曲的查找表；这种几何不是自动附赠的。

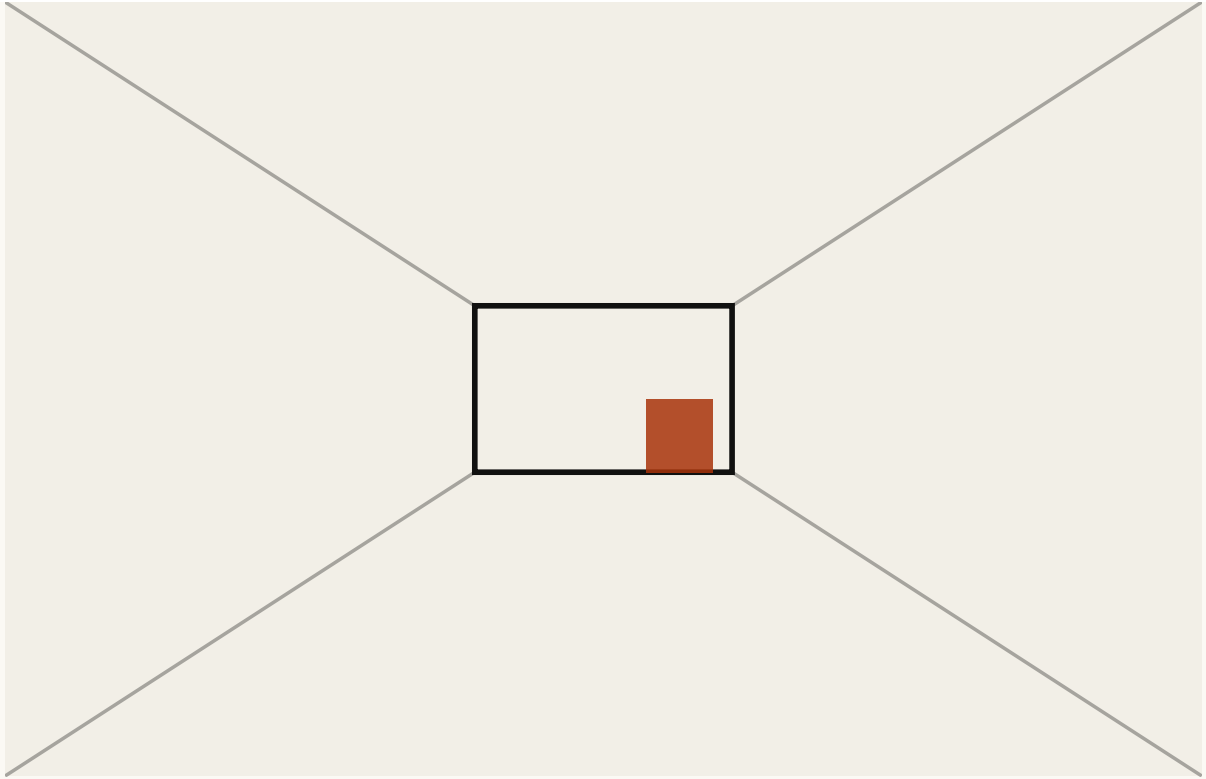
这意味着你可以在那里做一些在像素上根本讲不通的事。你可以问附近有什么。你可以朝某个方向走，看着世界沿着某种特定的方式平滑地变化。你可以拿两个房间，然后站在它们中间。



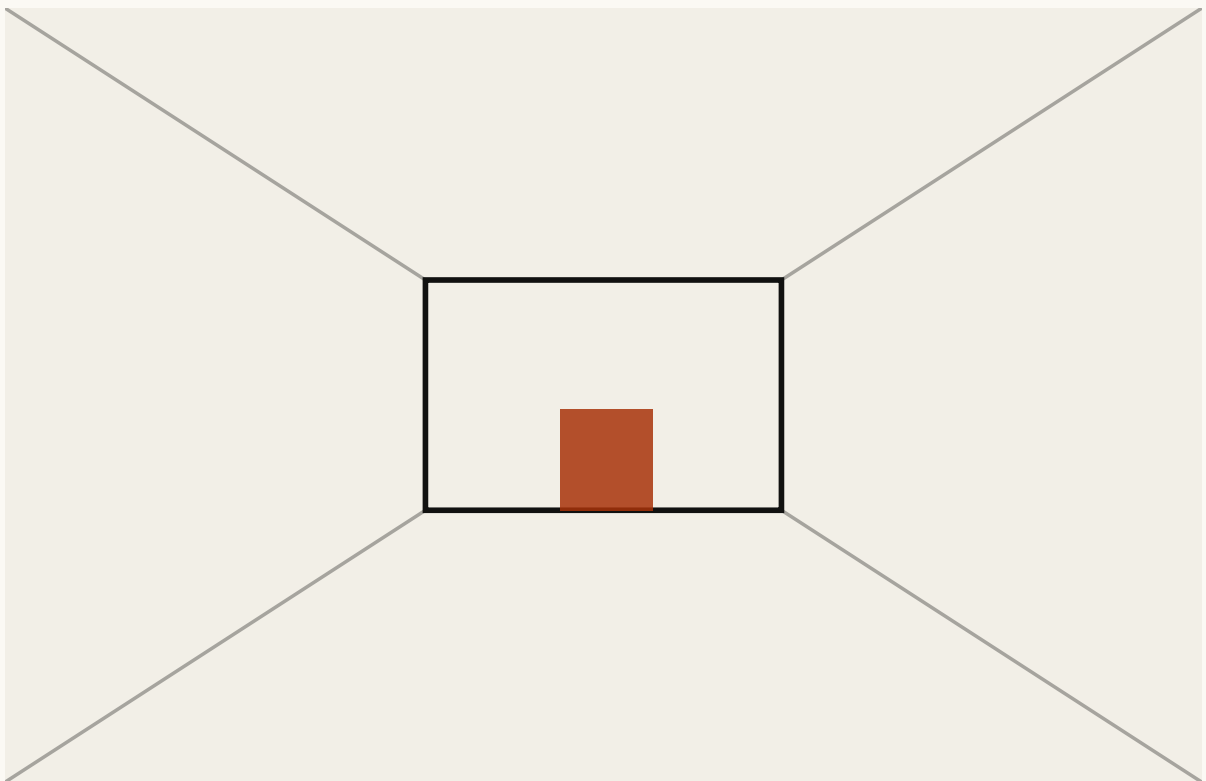
一个房间



把两张画面平均一下



另一个房间



把两个数字平均一下，再解码

混出来的画面是个房间吗？

不是。它是两个房间同时叠在一起，而那不是你走进去的東西。

解码出来的那个是房间吗？

是，滑块上每一处都是。那个空间里的每一个点都解码成一个房间。

FIG. 4.3 对这个刻意构造的解码器，两行都诚实。平均两张图，会得到两个房间淡淡地叠在一起；平均两个坐标再解码，则始终留在学出的「房间流形」上。真实的潜在空间只有在训练把样本之间

的区域也变得有意义之后，才配得到这种表现。

那幅图就是所有人费这个劲的理由。两张画面的中点是一个幽灵。两份描述的中点是一个房间。

一个「两个有效点之间的那个点本身也有效」的空间，是非常值钱的。它也正是像素给不了你的东西。预测、规划和搜索，全都涉及走到一个你没去过的地方。如果每一步落下去的都是没有意义的东西，那它们一个都跑不起来。

这也是为什么变分自编码器里的噪声不是个意外。刻意把每张画面落点弄模糊，会逼着相邻的点解码出相似的东西，而这正是让这个空间能被走通、而不是变成一堆互不相干的地址的原因。

什么样的才算好

小不是目标。一个数字非常小，也非常没用。

你想要的性质是：要紧的东西容易拿到，不要紧的东西已经没了。三个检验，大致按「它们有多常是真正出问题的那一个」排序：

它有没有留住未来所取决于的东西？如果两种情形会导向不同的结果，却落在同一个点上，那么后面的任何东西都没法把它们分开。这是补不回来的。被这道口子扔掉的，就是没了。

相邻的点意味着相邻的东西吗？一个「走一小步就可能落到任何地方」的空间，只是一张多绕了几道弯的查找表。平滑，才是让你能有目的地在里面移动的东西。

它容易往前推吗？一份描述可以完全忠实，同时又是个在时间上往前推的噩梦。这已经是你的问题。一开始要压缩，图的就是拿到一个能便宜地往前推的东西。

它的代价

重建画面是一个替身，而替身会跑偏。

重建这个检验问的是：这串短数字能不能把摄像机看到的東西再生成出来。而你真正想知道的是：它有没有留住这个决定所取决于的东西。这两件事会以一种很具体、也很讨厌的方式分开。占据像素最多的那些东西，通常不是要紧的那些。于是一份按「能不能重建像素」打分的描述，会把自己花在纹理和天气上，然后悄悄丢掉那个你需要避开的、又小又快的东西。

还有第二个代价，更隐蔽。这道口子丢掉的任何东西，从那一刻起就找不回来了。瓶颈之后的一切都只靠这串短数字工作。所以一个在这里、在很早的时候、由一个完全不知道接下来是什么任务的部件做出的选择，悄悄给它后面的一切定了上限。

把这么重要的一个决定放在那种位置，是件挺奇怪的事。这也是为什么这个领域的争论有那么大一部分是关于管子的中段，而不是它的两头。

试一试

1 摄像机每一帧交给你几万个数字，而决定要不要往前走只需要两三个。瓶颈是干什么用的？

- a. 把模型做小，让它跑得更快
- b. 逼着所有东西挤过一个窄口，于是活下来的正是画面真正由什么构成的
- c. 去掉摄像机的噪点
- d. 把文件在硬盘上压小

答案 b. 逼着所有东西挤过一个窄口，于是活下来的正是画面真正由什么构成的. 速度和体积都是副作用。之所以要挤，是因为想挤过去就必须把「一开始生成这张画面的那些东西」还原出来。

2 没有人手工挑选那串短数字里该装什么。为什么它最后还是装了对的东西？

- a. 架构里每个概念有一个单元
- b. 如果这些画面本来就由少数几样东西的变化产生，那么还原这几样就是重建它们最省事的办法
- c. 训练标注里写了
- d. 解码器被告知了答案

答案 b. 如果这些画面本来就由少数几样东西的变化产生，那么还原这几样就是重建它们最省事的办法. 这个排序是被压力逼出来的，不是设计出来的。这既是它吸引人的地方，也是为什么这个排序只是「大致」是你想要的那个。

3 把两个不同房间的两张画面平均一下。你会得到什么？

- a. 一个介于两者中间的房间
- b. 两个房间同时淡淡地叠在一起，那不是房间
- c. 第一个房间
- d. 一张空白画面

答案 b. 两个房间同时淡淡地叠在一起，那不是房间. 这就是像素空间里做混合的真实结果。它也是最能说明「为什么你想要一个『两个有效点之间也有效』的空间」的理由。

4 改成把两份简短描述平均一下，再解码。你会得到什么？

- a. 同一个幽灵
- b. 一个房间，因为那个空间里的每个点都解码成一个房间
- c. 什么都没有，这些数字加不起来
- d. 一张两个房间并排的画面

答案 b. 一个房间，因为那个空间里的每个点都解码成一个房间. 预测、规划和搜索都要走到没去过的地方。只有当中间那些位置是有意义的，这件事才成立。

5 变分自编码器里的噪声，为什么不只是个麻烦？

- a. 它让训练更快
- b. 它把训练数据藏起来
- c. 把每张画面的落点弄模糊，会逼着相邻的点解码出相似的东西
- d. 它减少了参数量

答案 c. 把每张画面的落点弄模糊，会逼着相邻的点解码出相似的东西. 平滑是让这个空间能被走通的那个性质。没有它，你手上就是一张多绕了几道弯的查找表。

6 以下哪一条**不是**「一份好的紧凑描述」的标准？

- a. 它留住了未来所取决于的东西
- b. 相邻的点意味着相邻的东西
- c. 它尽可能地小
- d. 它容易在时间上往前推

答案 c. 它尽可能地小. 一个数字非常小，也非常没用。小是「留住了对的东西」的结果，从来不是目标本身。

7 一份描述是按「它能多好地重建摄像机画面」来打分的。这会出什么问题？

- a. 不会出问题，这正是对的检验
- b. 它会把自己花在占像素最多的东西上，而那通常不是决定所取决于的东西
- c. 它会小到没法用
- d. 它会变得不可导

答案 b. 它会把自己花在占像素最多的东西上，而那通常不是决定所取决于的东西. 重建只是个替身。纹理和天气占了大部分像素；那个你需要避开的、又小又快的东西并不占。

8 为什么「瓶颈有多宽」是一个放置重要决定的尴尬位置？

- a. 它很难调
- b. 它后面的一切都只靠那串短数字，所以被丢掉的就没了，而丢它的时候还没人知道任务是什么
- c. 它占太多内存
- d. 它必须在训练前定好

答案 b. 它后面的一切都只靠那串短数字，所以被丢掉的就没了，而丢它的时候还没人知道任务是什么. 这个损失在下游补不回来。一个完全不知道接下来是什么任务的部件，悄悄给它后面的一切定了上限。

FIG. 4.4 八道关于这道收窄口和它产生的那个空间的题。最后两道讲的是那种容易点头同意、却很难一直记着的东西。

到这里为止的一切都是一次一个瞬间。一张画面进去，一份简短描述出来，还什么都没动过。

多谢阅读。第 5 章讲的是：当你开始把这份描述往前推的时候会发生什么。那套把「一个状态加一个动作」变成「下一个状态」的机器，以及你每让它推一次就会渗进来的那点误差。

参考资料

Challenging Common Assumptions in the Unsupervised Learning of Disentangled Representations

LOCATELLO ET AL., 2019 ↗

证明一个重要边界：没有归纳偏置，纯无监督的解耦在一般情况下不可识别。瓶颈会施加压力，却不会自动给坐标轴命名。

<https://arxiv.org/abs/1811.12359>

Auto-Encoding Variational Bayes KINGMA & WELLING, 2013 ↗

变分自编码器。它加进去的噪声，正是这个空间最后能被走通、而不是变成一堆互不相干的地址的原因。

<https://arxiv.org/abs/1312.6114>

Reducing the Dimensionality of Data with Neural Networks

HINTON & SALAKHUTDINOV, 2006 ↗

在有现代机器撑腰之前的瓶颈论证。需付费。

<https://doi.org/10.1126/science.1127647>

World Models HA & SCHMIDHUBER, 2018 ↗

编码器、潜在动力学、紧凑控制器，还包括把策略完全放在模型生成的环境里训练、再搬回真实环境的实验。

<https://arxiv.org/abs/1803.10122>

Learning Latent Dynamics for Planning from Pixels (PlaNet)

HAFNER ET AL., 2018 ↗

从图像里学出随机潜在动力学，然后在决策时对它做搜索。图 2.4 的真实版本。

<https://arxiv.org/abs/1811.04551>

Neural Discrete Representation Learning (VQ-VAE) VAN DEN OORD ET AL., 2017 ↗

当那份简短描述被强制变成少数几个离散符号、而不是连续数字时，会发生什么。

<https://arxiv.org/abs/1711.00937>

beta-VAE HIGGINS ET AL., 2017 ↗

把瓶颈上的压力调大，坐标轴就开始对上一些你叫得出名字的东西。这篇也很好地展示了那样做的代价。

<https://openreview.net/forum?id=Sy2fzU9gl>

Early Visual Concept Learning with Unsupervised Deep Learning

HIGGINS ET AL., 2016 ↗

「一个好的表征，是它的各个方向都有含义」这个主张，写在这个领域还不拥挤的时候。

<https://arxiv.org/abs/1606.05579>

Representation Learning: A Review and New Perspectives

BENGIO, COURVILLE & VINCENT, 2013 ↗

这篇综述梳理了「一个好的学出来的表征是拿来干什么的」，写在预测彻底赢下这场争论之前。

<https://arxiv.org/abs/1206.5538>

FIG. 4.5 排序是给要在这上面接着做事的人用的，而不是为了历史完整性。